Computer Architecture and Organization

# Lab 12 : Extending the CPU (BNZ, Stack, and Subroutines)

## Objectives

In this lab, you will extend the CPU from Lab 11 with a conditional branch on non-zero (BNZ), a hardware stack with PUSH and POP, and subroutine calls with CALL and RET. At the end, your CPU will run a program in which one subroutine calls another and registers are saved on the stack.

- Add BNZ by reusing the Type bit of the BZ opcode.
- Build a 6-bit stack pointer (SP) and the PUSH and POP instructions.
- Add CALL and RET, which store the return address on the stack.
- Write and run a program with nested subroutine calls.

## Requirements

- Working cpu.dig, controlUnit.dig, miniALU.dig and the other .dig files from Labs 9 to 11.
- Copy all Lab 11 files into a new folder named Lab12 and work only on the copies.
- Digital simulator (same version as Labs 9 to 11).

## Before You Start: Three Changes to Your Lab 11 Files

Make these changes first (or use the Lab 12 starting files provided by your TA):

1. In cpu.dig, rename the four switch inputs to SW0, SW1, SW2, SW3 (SW0 = least significant bit), matching the Lab 11 memory map.
2. Disconnect the programCounter LD pin from the Clock and connect it to a Supply voltage (Components → Wires → Supply voltage). The PC still loads every cycle, but a clock signal should only drive clock pins, never logic inputs.
3. Clear the programROM contents (right-click → Edit Content → set every word to 0).

## Instruction Set After This Lab

Instruction format (14 bits, unchanged): Opcode(3) + Type_Bit(1) + Reg1_Addr(3) + Reg2_Addr(3) + Address_Field(4).

| Opcode | Type | Instruction | Operation |
|---|---|---|---|
| **000** | 0 | LOAD Rd, Rs | Rd ← Rs |
| **000** | 1 | LOAD Rd, Mem[addr] | Rd ← Mem[addr] (addr 0 to 15) |
| **001** | X | STORE Rs, Mem[addr] | Mem[addr] ← Rs |
| **010** | 0 | ADD R1, R2 | R1 ← R1 + R2, updates zero flag |
| **010** | 1 | SUB R1, R2 | R1 ← R1 - R2, updates zero flag |
| **011** | 0 | BZ addr | PC ← addr if Zero_Flag = 1 |
| **011** | 1 | BNZ addr (new) | PC ← addr if Zero_Flag = 0 |
| **100** | X | BR addr | PC ← addr |
| **101** | X | LDI Rd, imm | Rd ← Address_Field (0 to 15) |
| **110** | 0 | PUSH Rs (new) | SP ← SP - 1, then Mem[SP] ← Rs |
| **110** | 1 | POP Rd (new) | Rd ← Mem[SP], then SP ← SP + 1 |
| **111** | 0 | CALL addr (new) | SP ← SP - 1, Mem[SP] ← PC + 1, PC ← addr |
| **111** | 1 | RET (new) | PC ← Mem[SP], then SP ← SP + 1 |

PUSH and POP use the Reg1_Addr field (bits 9 to 7) for the register. CALL uses the Address_Field. Fields that an instruction does not use must be 0.

Memory map:

| Address | Use | Access |
|---|---|---|
| Mem[0] to Mem[13] | Normal data RAM (variables) | LOAD, STORE |
| Mem[14] | 7-segment display port | STORE only |
| Mem[15] | Switch port (SW3 to SW0) | LOAD only |
| Mem[16] to Mem[63] | Stack (grows downward from Mem[63]) | PUSH, POP, CALL, RET (through SP) |

**Program limit for this lab:** a Data RAM word is only 4 bits wide, so CALL can store only the lower 4 bits of the return address. Keep every program within ROM addresses 0 to 15, and never place a CALL at address 15 (you will explain why in Q5).

New and changed control signals:

| Signal | Is 1 for | Purpose |
|---|---|---|
| **StackOp** | PUSH, POP, CALL, RET | Updates SP, selects the stack address for dataRAM, disables the I/O ports |
| **Push** | PUSH, CALL | Selects SP - 1 (decrement) instead of SP + 1 |
| **DinSel** | CALL | dataRAM.Din takes the return address instead of a register |
| **PCSource (now 2 bits)** | - | 00 = PC + 1, 01 = Address_Field_Ext (BR, taken BZ/BNZ, CALL), 10 = return address (RET) |

## Lab 12.1 : Branch if Not Zero (BNZ)

Until now, BZ ignored its Type bit. BNZ reuses opcode 011 with Type = 1. The branch is taken when (Type = 0 and Zero_Flag = 1) or (Type = 1 and Zero_Flag = 0), which is exactly Type XOR Zero_Flag.

### 1. Update controlUnit

1.1. Open controlUnit.dig. Find the AND gate that computes BZ AND Zero_Flag (its output goes to the PCSource OR gate).

1.2. Add an XOR gate (Components → Logic → XOr). Connect Type and Zero_Flag to its inputs. Label its output TxZ.

1.3. Disconnect Zero_Flag from the BZ AND gate and connect TxZ instead. The AND output is now Taken = BZ AND (Type XOR Zero_Flag).

1.4. PCSource = Taken OR BR stays as it is.

### 2. Update the Test component in controlUnit.dig

2.1. Add these two rows after the row that starts with 0b011 0 1:

```
0b011 1 0 0 0 0 0 1 0b00 0
0b011 1 1 0 0 0 0 0 0b00 0
```

2.2. Delete the two rows that start with 0b110 and 0b111. These opcodes are no longer unused; they get their own tests in Lab 12.2 and Lab 12.3.

2.3. Run the tests. All rows must pass.

### 3. Test program 1 (countdown with BNZ)

| Addr | Instruction | Encoding | Note |
|---|---|---|---|
| 0 | LDI R1, 3 | 0x2883 | R1 ← 3 |
| 1 | LDI R2, 1 | 0x2901 | R2 ← 1 |
| 2 | STORE R1, Mem[14] | 0x088E | Show R1 |
| 3 | SUB R1, R2 | 0x14A0 | R1 ← R1 - 1 |
| 4 | BNZ 2 | 0x1C02 | Loop while R1 ≠ 0 |
| 5 | STORE R1, Mem[14] | 0x088E | Show 0 |
| 6 | BR 6 | 0x2006 | Stop |

Single-step the program and verify that the display shows 3, 2, 1, 0 and the PC stays at 6.

**Q1:** Write the four cases of Type and Zero_Flag for opcode 011. Show that Type XOR Zero_Flag gives the correct branch decision for both BZ and BNZ.

**TA Check: _____ Time: _____**

## Lab 12.2 : Stack Pointer, PUSH and POP

The stack is full descending, the same model as A32 PUSH and POP: SP points to the last item pushed, and the stack grows toward lower addresses. SP is a 6-bit register that starts at 0. PUSH first decrements SP and then writes, so the first PUSH writes Mem[63]. POP first reads and then increments SP.

| Instruction | dataRAM address | Data moved | SP after the clock |
|---|---|---|---|
| **PUSH Rs** | SP - 1 | Mem[SP - 1] ← Rs | SP - 1 |
| **POP Rd** | SP | Rd ← Mem[SP] | SP + 1 |

### 1. Update controlUnit

1.1. On the decoder, label y6 = PP (PUSH/POP) and y7 = CR (CALL/RET).

1.2. Add a 2-input OR gate: StackOp = PP OR CR. Add an Output labelled StackOp and connect it.

1.3. Add a 2-input AND gate: Push = StackOp AND (NOT Type). Use the NOT Type signal you already built for sel1 in Lab 10. Add an Output labelled Push. (Push is 1 for PUSH and for CALL.)

1.4. Add a 2-input AND gate: POP = PP AND Type. This is an internal signal, not an output.

1.5. Change the RegWrite OR gate to 4 inputs and connect POP: RegWrite = LOAD OR ADD OR LDI OR POP.

1.6. Change the sel0 OR gate (Lab 11.2 step 2.3) to 3 inputs and connect POP: sel0 = (LOAD AND Type) OR LDI OR POP. POP therefore selects RegWriteSource = 01 (memory).

1.7. MemRead: add a 2-input AND gate StackRead = StackOp AND Type and a 2-input OR gate MemRead = (LOAD AND Type) OR StackRead. Connect the OR output to the MemRead Output instead of the old connection.

1.8. MemWrite: add a 2-input OR gate MemWrite = STORE OR Push. Connect it to the MemWrite Output instead of the direct wire from STORE.

1.9. Add a second Test component, label it Stack, and enter these rows. Run all tests; all rows must pass.

```
Opcode Type Zero_Flag RegWrite MemRead MemWrite RegWriteSource StackOp Push
0b110 0 0 0 0 1 0b00 1 1
0b110 1 0 1 1 0 0b01 1 0
0b111 0 0 0 0 1 0b00 1 1
0b111 1 0 0 1 0 0b00 1 0
0b000 1 0 1 1 0 0b01 0 0
```

```
0b001 0 0 0 0 1 0b00 0 0
```

***Note:*** *The new Outputs appear as new pins at the bottom of the controlUnit symbol in cpu.dig. After reloading controlUnit, check that no existing wire runs underneath the new pins. If one does, move that wire, or it will be connected to the new pin.*

### 2. Build the stack pointer in cpu.dig

Tip: Tunnels (Components → Wires → Tunnel) keep the drawing tidy. Two tunnels with the same Net Name are connected.

2.1. Add a Register (Data Bits = 6) and label it SP. Connect C to the master clock and en to StackOp.

2.2. Add an Adder (Data Bits = 6). Connect a to SP.Q, b to a Constant Value 63 (Data Bits = 6), and the carry-in to Ground. Label the sum output SPm1. Adding 63 subtracts 1, because 63 is -1 in 6-bit two's complement.

2.3. Add a second Adder (Data Bits = 6). Connect a to SP.Q, b to a Constant Value 1 (Data Bits = 6), and the carry-in to Ground. Label the sum output SPp1.

2.4. Add a Multiplexer (Data Bits = 6, Selector Bits = 1): in_0 = SPp1, in_1 = SPm1, sel = Push. Connect its output to SP.D. This is the next value of SP.

2.5. Add a second Multiplexer (Data Bits = 6, Selector Bits = 1): in_0 = SP.Q, in_1 = SPm1, sel = Push. Label its output StackAddr. This is the address a stack instruction uses.

### 3. Select the dataRAM address

3.1. Delete the wire from Address_Field_Ext to dataRAM.A. Keep the Address_Field_Ext wire to the PC multiplexer.

3.2. Add a Multiplexer (Data Bits = 6, Selector Bits = 1): in_0 = Address_Field_Ext, in_1 = StackAddr, sel = StackOp. Connect its output to dataRAM.A.

### 4. Disable the I/O ports during stack instructions

4.1. The address decode from Lab 11.3 looks at the instruction's Address_Field, but stack instructions do not access Mem[Address_Field]. Without a fix, a stack instruction could trigger the display or the switch port.

4.2. Change the IO_High AND gate (Lab 11.3 step 1.2) to 4 inputs. Add a NOT gate with StackOp as its input and connect its output to the fourth input: IO_High = A3 AND A2 AND A1 AND (NOT StackOp). DispSel, SwSel and DispWrite are now 0 whenever StackOp = 1.

### 5. POP data path

5.1. Nothing new is needed. POP selects in_1 of the write-back multiplexer (MemOrSwData), and because SwSel = 0 during a stack instruction, that input carries the dataRAM value.

### 6. Test program 2 (swap two registers with the stack)

| Addr | Instruction | Encoding | Note |
|---|---|---|---|
| 0 | LDI R1, 7 | 0x2887 | R1 ← 7 |
| 1 | LDI R2, 2 | 0x2902 | R2 ← 2 |
| 2 | PUSH R1 | 0x3080 | Mem[63] ← 7, SP = 63 |
| 3 | PUSH R2 | 0x3100 | Mem[62] ← 2, SP = 62 |
| 4 | POP R1 | 0x3480 | R1 ← Mem[62] |
| 5 | POP R2 | 0x3500 | R2 ← Mem[63] |
| 6 | STORE R1, Mem[14] | 0x088E | Show R1 |

| Addr | Instruction | Encoding | Note |
|---|---|---|---|
| 7 | BR 7 | 0x2007 | Stop |

Add a Probe on SP.Q. Single-step the program and verify: R1 = 2, R2 = 7, the display shows 2, SP = 0, Mem[63] = 7 and Mem[62] = 2.

**Q2:** SP starts at 0. Why does the first PUSH write to Mem[63]?

**Q3:** Write the values of SP, Mem[62] and Mem[63] after each of the instructions at addresses 2 to 5. Why did R1 and R2 swap?

**TA Check: _____ Time: _____**

## Lab 12.3 : Subroutine Call and Return (CALL, RET)

CALL pushes the return address (PC + 1) and jumps to the Address_Field. RET pops the return address into the PC. A dataRAM word is 4 bits, so CALL stores only the lower 4 bits of PC + 1, and RET fills the upper 2 bits of the PC with 0.

### 1. Update controlUnit

1.1. Add a 2-input AND gate: DinSel = CR AND (NOT Type). Add an Output labelled DinSel. (DinSel is 1 only for CALL.)

1.2. Add a 2-input AND gate: RET = CR AND Type.

1.3. PCSource becomes 2 bits. Change the PCSource OR gate to 3 inputs and connect DinSel: PC0 = Taken OR BR OR DinSel. The second bit is PC1 = RET.

1.4. Add a Splitter/Merger (Input Splitting = 1,1, Output Splitting = 2). Connect PC0 to the top input and PC1 to the bottom input. Connect its output to the PCSource Output, and set that Output to Data Bits = 2.

1.5. Add a third Test component, label it Call, and enter these rows. Run all three tests; all rows must pass. (The first Test still passes, because PCSource values 0 and 1 equal 0b00 and 0b01.)

```
Opcode Type Zero_Flag PCSource DinSel StackOp Push MemWrite MemRead
0b111 0 0 0b01 1 1 1 1 0
0b111 0 1 0b01 1 1 1 1 0
0b111 1 0 0b10 0 1 0 0 1
0b111 1 1 0b10 0 1 0 0 1
0b100 0 0 0b01 0 0 0 0 0
0b110 0 0 0b00 0 1 1 1 0
```

### 2. Update the datapath in cpu.dig

2.1. Right-click the PC multiplexer and set Selector Bits = 2. Changing the selector bits moves its pins, so reconnect every wire: in_0 = PC + 1 (Adder output), in_1 = Address_Field_Ext, in_2 = RetAddr (step 2.2), in_3 = Constant Value 0 (Data Bits = 6), sel = PCSource (now 2 bits). The output still goes to programCounter IN.

2.2. Add a Splitter/Merger (Input Splitting = 4,2, Output Splitting = 6). Connect dataRAM.D to the top input and a Constant Value 0 (Data Bits = 2) to the bottom input. Label the output RetAddr.

2.3. Add a Splitter/Merger (Input Splitting = 6, Output Splitting = 4,2). Connect the PC + 1 Adder output to its input. Label the first output (bits 3 to 0) RetLow; leave the second output unconnected.

2.4. Delete the wire from registerFile.Da to dataRAM.Din. Keep Da connected to miniALU.A and to the display register D.

2.5. Add a Multiplexer (Data Bits = 4, Selector Bits = 1): in_0 = registerFile.Da, in_1 = RetLow, sel = DinSel. Connect its output to dataRAM.Din.

### 3. Test program 3 (call a subroutine that doubles R1)

| Addr | Instruction | Encoding | Note |
|---|---|---|---|
| 0 | LDI R1, 4 | 0x2884 | R1 ← 4 |
| 1 | CALL 6 | 0x3806 | Mem[63] ← 2, SP = 63, PC ← 6 |
| 2 | STORE R1, Mem[14] | 0x088E | Show R1 after the return |
| 3 | BR 3 | 0x2003 | Stop |
| 4, 5 | (empty) | 0x0000 | - |
| 6 | ADD R1, R1 | 0x1090 | Subroutine: R1 ← 2 × R1 |
| 7 | RET | 0x3C00 | PC ← Mem[63] = 2, SP = 0 |

Single-step the program and verify the PC sequence 0, 1, 6, 7, 2, 3 and the final values R1 = 8, display = 8, SP = 0 and Mem[63] = 2.

**Q4:** In Lab 12.2 step 4 you disabled the I/O ports during stack instructions. Without that change, what would the instruction CALL 14 do to the 7-segment display? Explain using DispSel and MemWrite.

**Q5:** Why must programs in this lab stay within ROM addresses 0 to 15? What goes wrong if a CALL instruction is placed at address 15?

**TA Check: _____ Time: _____**

## Lab 12.4 : Exercise - Nested Subroutines

Write a program with a main part and two subroutines, SHOW and COUNT:

- **Main:** read N (0 to 15) from the switches into R1. Put the value 9 in R2; the main part needs this value again at the end. CALL SHOW to display N. Then CALL COUNT repeatedly while R1 is not 0 (use BNZ). Finally, STORE R2 to Mem[13] and stop (BR to its own address).
- **SHOW:** display R1 (STORE R1, Mem[14]) and return.
- **COUNT:** save R2 with PUSH, use R2 as the constant 1 to decrement R1 with SUB, CALL SHOW, restore R2 with POP, and return.

The whole program must fit in ROM addresses 0 to 15. Run it with N = 3 and N = 5: the display must count down to 0, and Mem[13] must be 9 at the end.

**Q6:** How many instructions does your program contain? For N = 3, how many clock cycles pass from address 0 until the stop instruction is first reached?

**Q7:** In the main part, BNZ comes right after CALL COUNT returns, but the SUB that sets the zero flag is inside COUNT. Why does BNZ still see the correct flag? Which instructions change Zero_Flag?

**Q8:** What is the largest number of entries on the stack while your program runs? At that moment, what does each entry hold, and what is SP?

**Q9:** In ARM A32, BL saves the return address in LR, not on the stack. Why must a function that calls another function still execute PUSH {LR}? Compare this with our CALL instruction.

**TA Check: _____ Time: _____**

## Submission

- controlUnit.dig, cpu.dig, and every other .dig file you changed (include the ROM and RAM contents you configured).
- Video record of the Lab 12.4 exercise running with N = 3.
- Written answers to Q1 to Q9.

Submission at: